



# NGUYÊN LÝ HỆ ĐIỀU HÀNH

## Phần 4: Lập lịch CPU

**NGUYỄN THỊ HẬU**

Khoa Công Nghệ Thông Tin  
Đại học Công Nghệ - Đại học quốc gia Hà Nội

# Phần 4: Lập lịch CPU

- Khái niệm
- Các tiêu chí lập lịch
- Thuật toán lập lịch
- Các phương pháp đánh giá thuật toán lập lịch

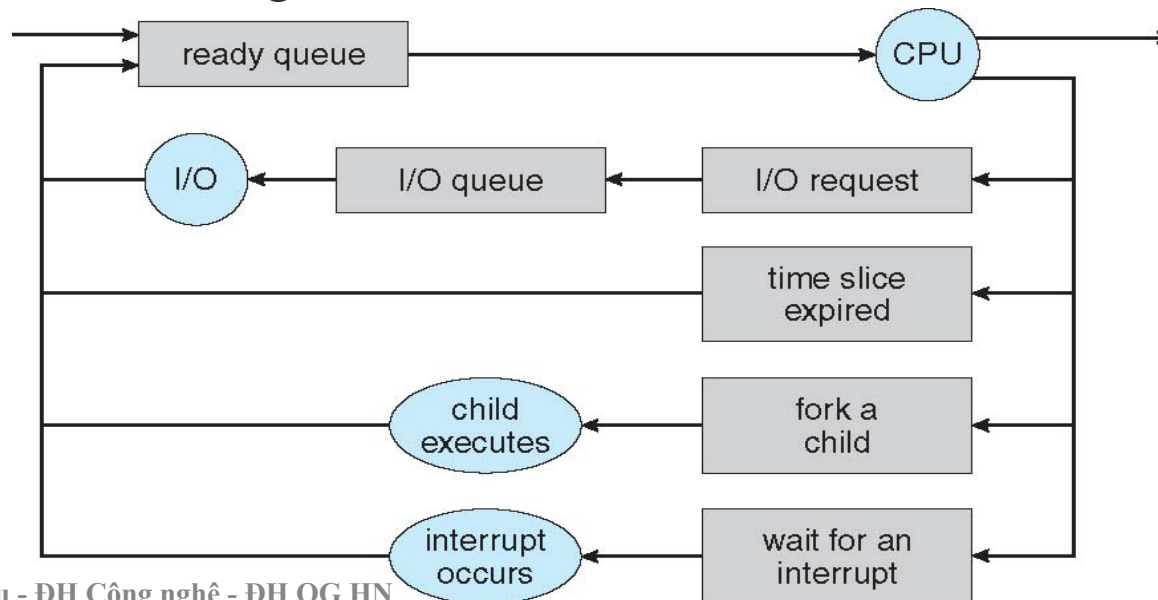
# Phần 4: Lập lịch CPU

## 4.1 Khái niệm & các tiêu chí lập lịch

# Khái niệm

## ■ Lập lịch CPU

- Nhằm lựa chọn giữa các tiến trình/luồng đã sẵn sàng chạy để cấp phát CPU cho nó
- Hoạt động khi
  - Ngắt phần cứng / trường hợp ngoại trừ của phần mềm
  - Gọi hàm hệ thống



# Các phương pháp lập lịch

- **Không cho phép dừng (Nonpreemptive):** Bộ lập lịch hoạt động khi tiến trình đang chạy kết thúc hoặc không thể chạy tiếp
  - Ví dụ: Microsoft Windows 3.x
    - Một tiến trình chuyển từ trạng thái running sang waiting
    - Một tiến trình chuyển từ trạng thái running sang ready
    - Một tiến trình chuyển từ trạng thái waiting sang ready
    - Một tiến trình kết thúc
  - Không yêu cầu phần cứng đặc biệt nào, vd timer
- **Cho phép dừng (Preemptive):** Bộ lập lịch hoạt động bất cứ khi nào có thể
  - Ví dụ: Windows 95 trở về sau, Mac OS X

# Bộ điều phối (dispatcher)

- Trao quyền điều khiển CPU cho các tiến trình được lựa chọn bởi bộ lập lịch CPU:
  - Chuyển trạng thái (context switch)
  - Chuyển về chế độ người dùng
  - Thực hiện tiến trình theo trạng thái đã lưu
- Thời gian cần để bộ điều phối dừng một tiến trình và thực hiện tiến trình khác gọi là **độ trễ (latency)** của bộ điều phối

# Các tiêu chí của bộ lập lịch

1. Tối đa thời gian sử dụng CPU
2. Tối đa hoá số lượng tiến trình hoàn thành trong một đơn vị thời gian
3. Giảm thiểu thời gian quay vòng – thời gian để thực thi một tiến trình
4. Giảm thiểu thời gian phản hồi khi có yêu cầu
5. Giảm thiểu thời gian chờ đợi CPU trong hàng đợi sẵn sàng

# Phần 4: Lập lịch CPU

## 4.2 Các thuật toán lập lịch



# FCFS (First Come First Served)

- Tiến trình nào có yêu cầu sử dụng CPU trước sẽ được thực hiện trước
- **Ưu điểm:** Thuật toán đơn giản nhất
- **Nhược điểm:** Hiệu quả của thuật toán phụ thuộc vào *thứ tự* của các tiến trình trong hàng chờ, vì thứ tự này ảnh hưởng rất lớn đến *thời gian chờ trung bình* (*average waiting time*)

# FCFS (First Come First Served)

Tiến trình Thời gian chạy

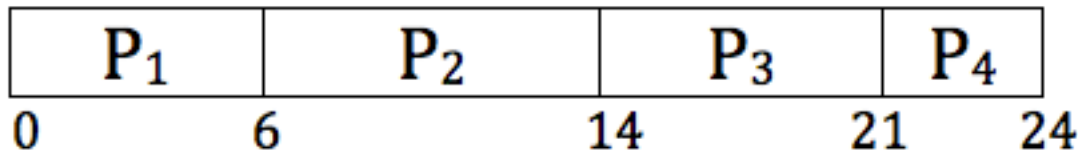
$P_1$  6

$P_2$  8

$P_3$  7

$P_4$  3

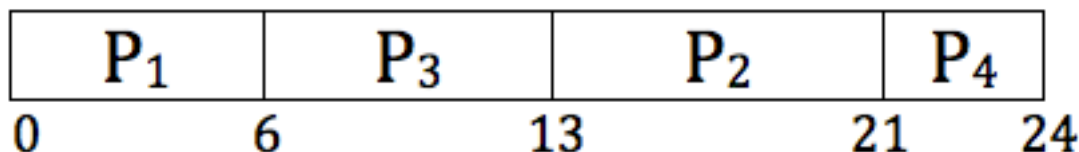
- Giả sử thứ tự các tiến trình trong hàng chờ là:  $P_1, P_2, P_3, P_4$   
Biểu đồ Gantt cho bộ lập lịch FCFS:



- Thời gian chờ  $P_1 = 0$ ;  $P_2 = 6$ ;  $P_3 = 14$ ;  $P_4 = 21$
- Thời gian chờ trung bình:  $(0 + 6 + 14 + 21)/4 = 10.25$

Giả sử các tiến trình có thứ tự là:  $P_1, P_3, P_2, P_4$

■ Biểu đồ Gantt:



■ Thời gian chờ  $P_1 = 0$ ;  $P_2 = 6$ ;  $P_3 = 13$ ;  $P_4 = 21$

■ Thời gian chờ trung bình:  $(0 + 6 + 13 + 21)/4 = 10 < 10.25$

■ Hiện tượng “đoàn hộ tống” (convoy effect) *Res. pend time*

- Tiến trình có thời gian chạy ngắn chờ sau tiến trình có thời gian chạy dài
- CPU và các thiết bị vào/ra có nhiều thời gian rỗi, và thời gian chờ trung bình của các tiến trình lâu

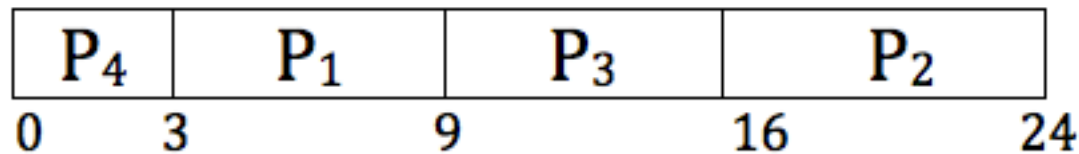
# Shortest-Job-First (SJF)

12

- Mỗi tiến trình có thêm tham số *độ dài của phiên sử dụng CPU tiếp theo*  $t_{nextburst}$
- Tiến trình có  $t_{nextburst}$  nhỏ nhất sẽ được lập lịch sử dụng CPU trước
- Nếu hai tiến trình có  $t_{nextburst}$  bằng nhau thì FCFS được áp dụng

# Shortest-Job-First (SJF)

- Với SJF, biểu đồ Gantt cho 4 tiến trình:



Thời gian chờ trung bình:  $(0 + 3 + 9 + 16)/4 = 7$

- SJF cho phép giảm thiểu thời gian chờ trung bình

- Khó khăn của thuật toán SJF là dự đoán *độ dài của phiên sử dụng CPU tiếp theo*
- SJF thường không được sử dụng cho bộ lập lịch ngắn hạn
- Dự đoán *độ dài của phiên sử dụng CPU tiếp theo* thông qua **hàm trung bình lũy thừa** (exponential averaging)
- Vd *Preemptive SJF (shortest-remaining-time-first)*

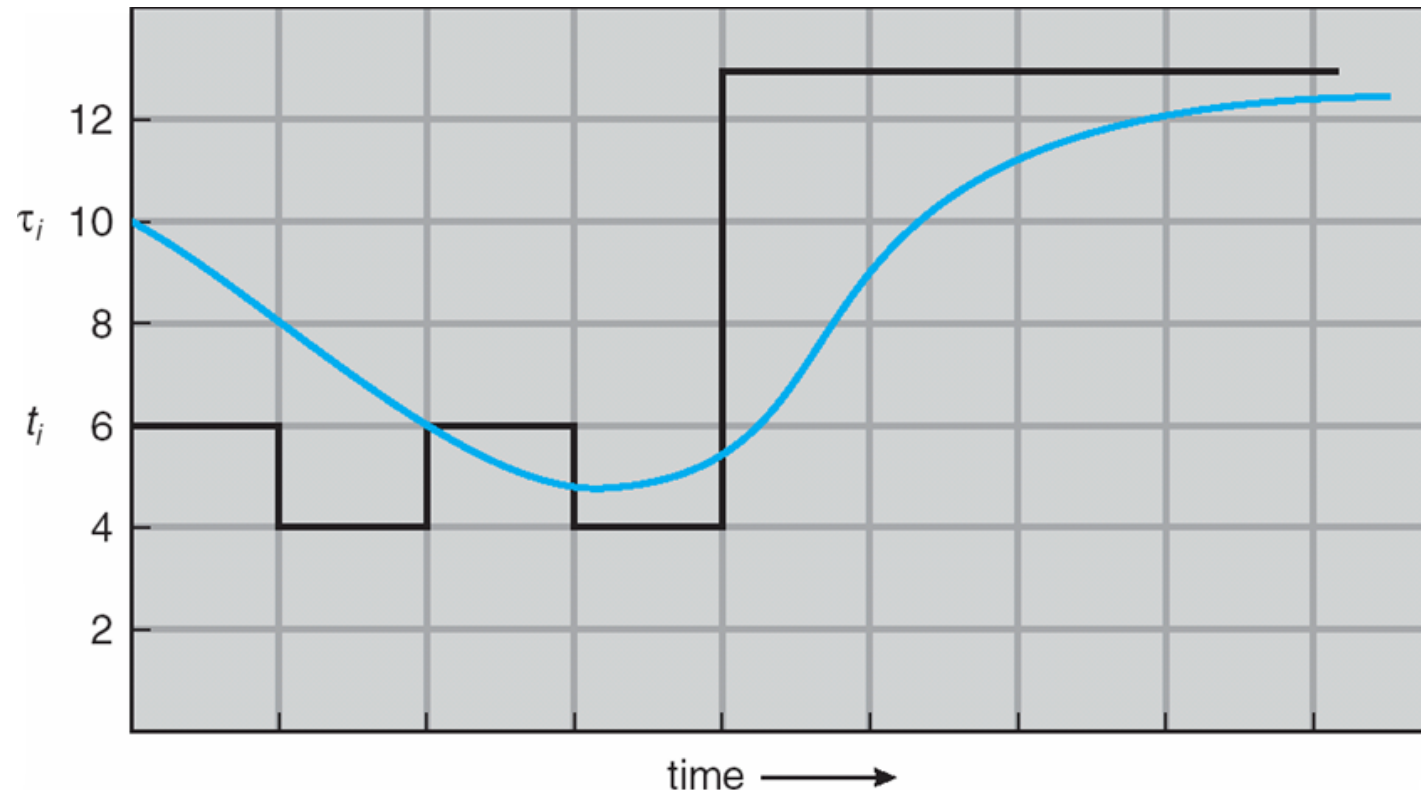
$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n \quad (0 \leq \alpha < 1)$$

$t_i$ : độ dài của phiên sử dụng thứ  $i$

$\tau_i$ : dự đoán độ dài của phiên tiếp theo

$$\begin{aligned} \text{Hay } \tau_{n+1} &= \alpha t_n + (1 - \alpha) \alpha t_{n-1} + \dots \\ &\quad + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ &\quad + (1 - \alpha)^{n+1} \tau_0 \end{aligned}$$

# Dự đoán độ dài của phiên sử dụng CPU tiếp theo

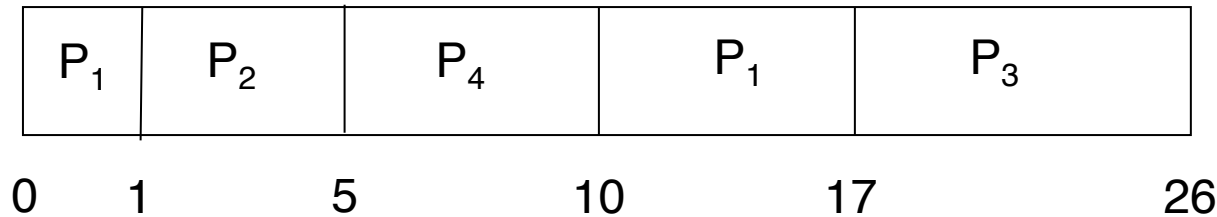


|                      |    |   |   |   |   |    |    |    |     |
|----------------------|----|---|---|---|---|----|----|----|-----|
| CPU burst ( $t_i$ )  |    | 6 | 4 | 6 | 4 | 13 | 13 | 13 | ... |
| "guess" ( $\tau_i$ ) | 10 | 8 | 6 | 6 | 5 | 9  | 11 | 12 | ... |

## Ví dụ *Preemptive SJF*

| Process | Thời gian đến | Thời gian chạy |
|---------|---------------|----------------|
| $P_1$   | 0             | 8              |
| $P_2$   | 1             | 4              |
| $P_3$   | 2             | 9              |
| $P_4$   | 3             | 5              |

### ■ Biểu đồ Gantt của *Preemptive SJF*



■ Thời gian chờ trung bình của *Preemptive SJF* =  $[(10-1)+(1-1)+(17-2)+5-3]/4 = 26/4 = 6.5$

■ Thời gian chờ trung bình của *Non Preemptive SJF* =  $(0 + 7 + 9 + 15)/4 = 7.75$



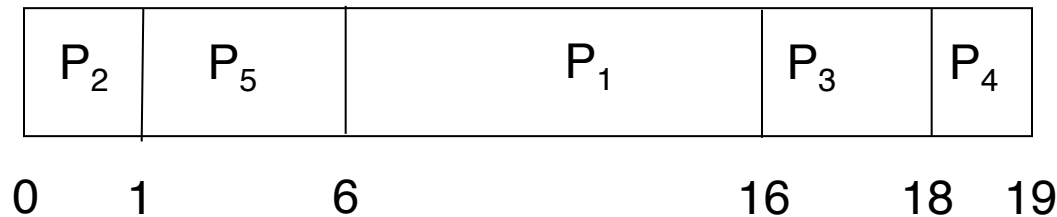
# Lập lịch ưu tiên

- Mỗi tiến trình có một số hiệu ưu tiên
- Cấp phát CPU cho tiến trình có ưu tiên cao nhất/bé nhất
- SJF là trường hợp đặc biệt của lập lịch ưu tiên với số hiệu ưu tiên là nghịch đảo của *độ dài của phiên sử dụng CPU tiếp theo*

## Ví dụ lập lịch ưu tiên

| Process | Thời gian chạy | Thứ tự ưu tiên |
|---------|----------------|----------------|
| $P_1$   | 10             | 3              |
| $P_2$   | 1              | 1              |
| $P_3$   | 2              | 4              |
| $P_4$   | 1              | 5              |
| $P_5$   | 5              | 2              |

### ■ Biểu đồ Gantt của lập lịch ưu tiên



### ■ Thời gian chờ trung bình = 8.2

# Lập lịch Round Robin (RR)

- Áp dụng cho hệ thống chia sẻ thời gian
- Mỗi tiến trình chiếm CPU trong một đơn vị thời gian (**time quantum**  $q \sim 10-100$  msec). Sau khoảng thời gian này, tiến trình bị dừng lại, xếp hàng chờ ở cuối hàng đợi sẵn sàng.
- Nếu có  $n$  tiến trình trong hàng đợi sẵn sàng thì không tiến trình nào phải đợi lâu hơn  $(n-1)q$  đơn vị thời gian
  - Nếu  $q$  lớn: giống lập lịch FCFS
  - Nếu  $q$  nhỏ: mất nhiều thời gian chuyển trạng thái

# Ví dụ RR với Time Quantum = 4

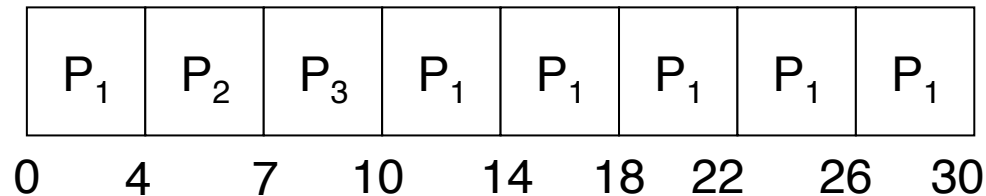
Process      Thời gian chạy

$P_1$                       24

$P_2$                       3

$P_3$                       3

■ Biểu đồ Gantt:

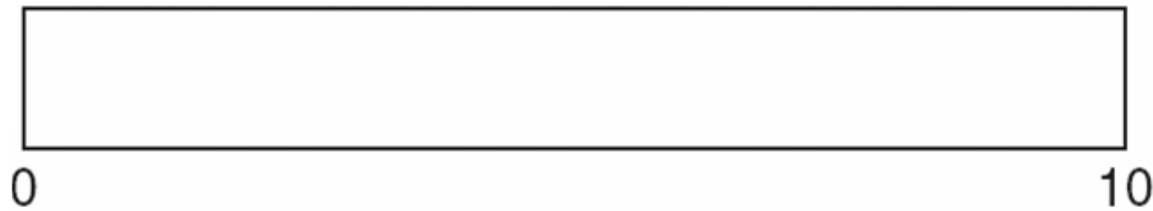


- Thời gian hoàn thành trung bình  ~~$30/3 = 10$~~
- Thời gian chờ trung bình  $((10-4) + 4 + 7)/3 = 17/3 > \text{lập lịch SJF}$
- Tuy nhiên thời gian phản hồi yêu cầu nhanh hơn của SJF

# Time Quantum vs Context Switch Time

21

process time = 10

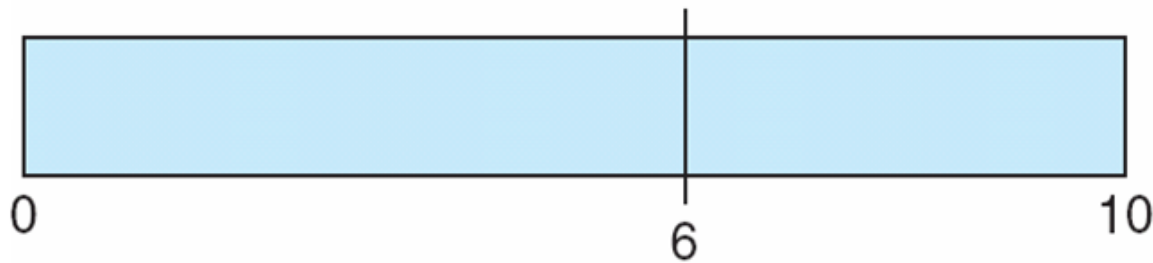


quantum

12

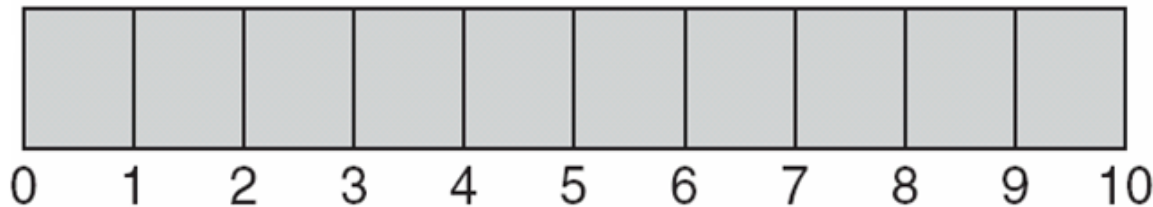
context  
switches

0



6

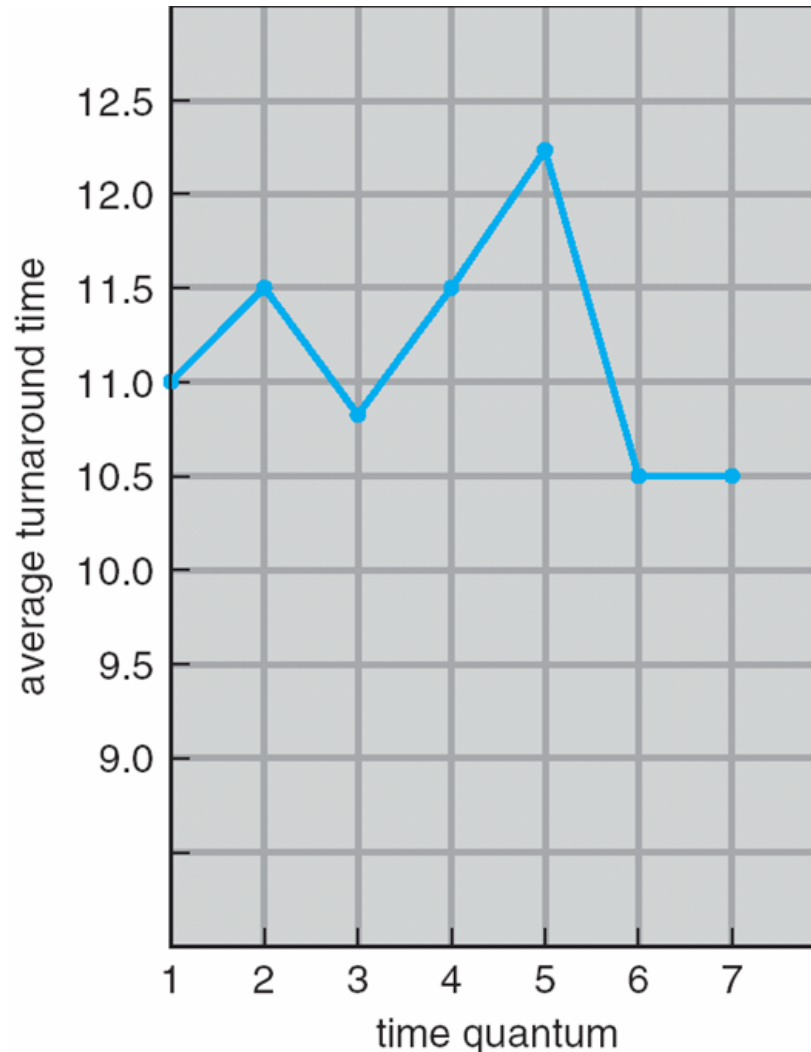
1



1

9

# Thời gian hoàn thành biến đổi theo Time Quantum



| process | time |
|---------|------|
| $P_1$   | 6    |
| $P_2$   | 3    |
| $P_3$   | 1    |
| $P_4$   | 7    |

**80% thời gian chạy CPU  
nên nhỏ hơn  $q$**

# Lập lịch hàng đợi đa cấp

- Hàng đợi sẵn sàng được chia ra nhiều hàng đợi. Vd: hàng đợi trước, hàng đợi sau
- Mỗi tiến trình chỉ ở một hàng đợi được sắp xếp từ trước
- Mỗi hàng đợi có riêng thuật toán lập lịch
  - Hàng đợi trước: RR
  - Hàng đợi sau: FCFS
- Cần có thuật toán lập lịch giữa các hàng đợi
  - Lập lịch ưu tiên: vd chạy các tiến trình ở hàng đợi trước rồi đến các tiến trình ở hàng đợi sau
  - Mỗi hàng đợi chiếm giữ CPU trong một khoảng thời gian nhất định

# Lập lịch hàng đợi phản hồi đa cấp

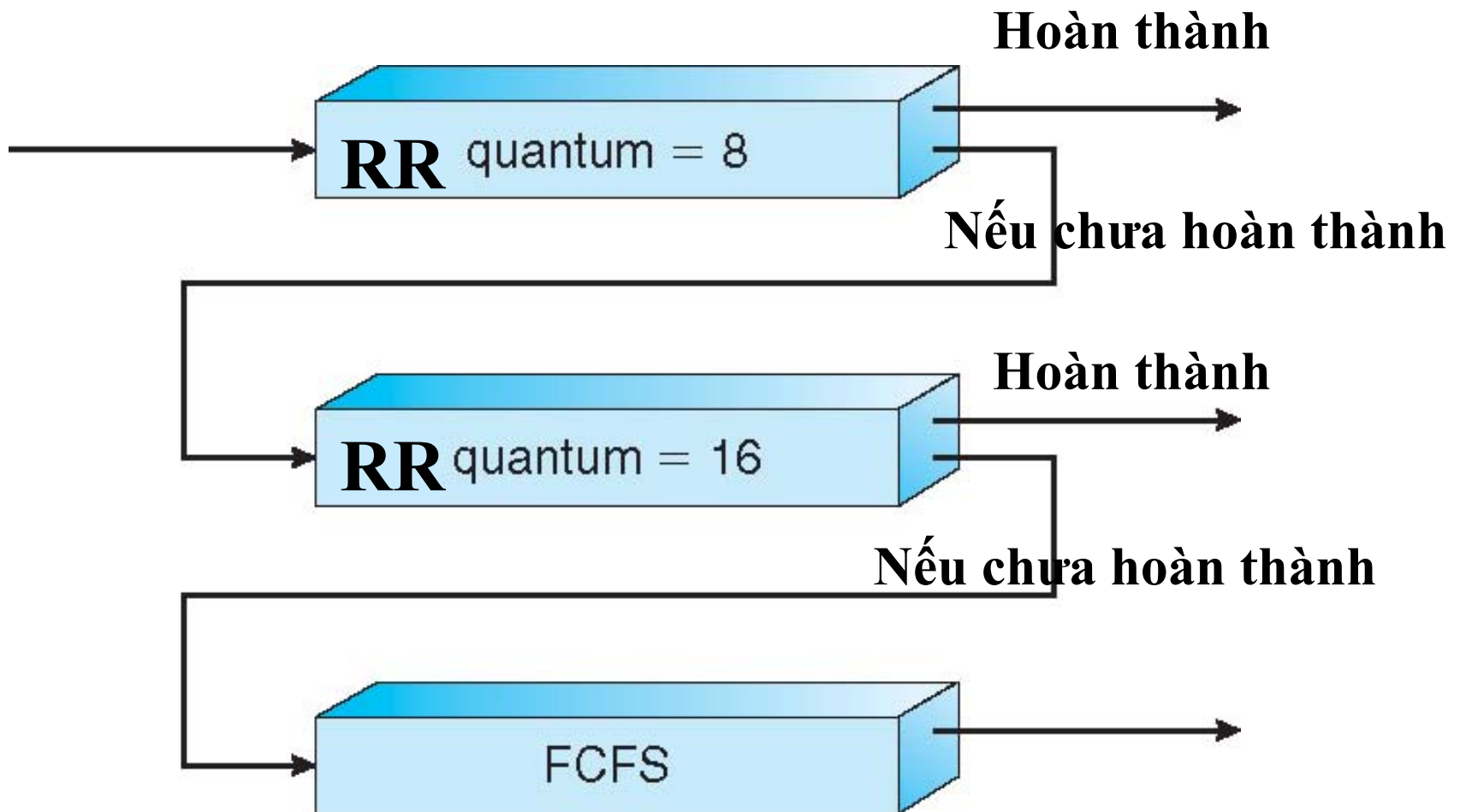
24

- Tiến trình có thể di chuyển giữa các hàng đợi
- Sử dụng các tham số sau:
  - Số lượng hàng đợi
  - Thuật toán cho mỗi hàng đợi
  - Phương thức chuyển tiến trình lên hàng đợi có ưu tiên cao hơn và ngược lại
  - Phương thức lựa chọn hàng đợi cho một tiến trình khi có yêu cầu



# VD Lập lịch hàng đợi phản hồi đa cấp

25



# Phần 4: Lập lịch CPU

## 4.4 Các phương pháp đánh giá thuật toán lập lịch

# Mô hình xác định (Deterministic Modeling)

- Dựa vào trình tự của các tiến trình, thời gian chạy CPU, thiết bị vào ra được giả định trước
  - Thường dựa vào trung bình hàm lũy thừa
  - Tính thời gian hoàn thành, thời gian sử dụng, thời gian chờ trung bình

# Định luật Little

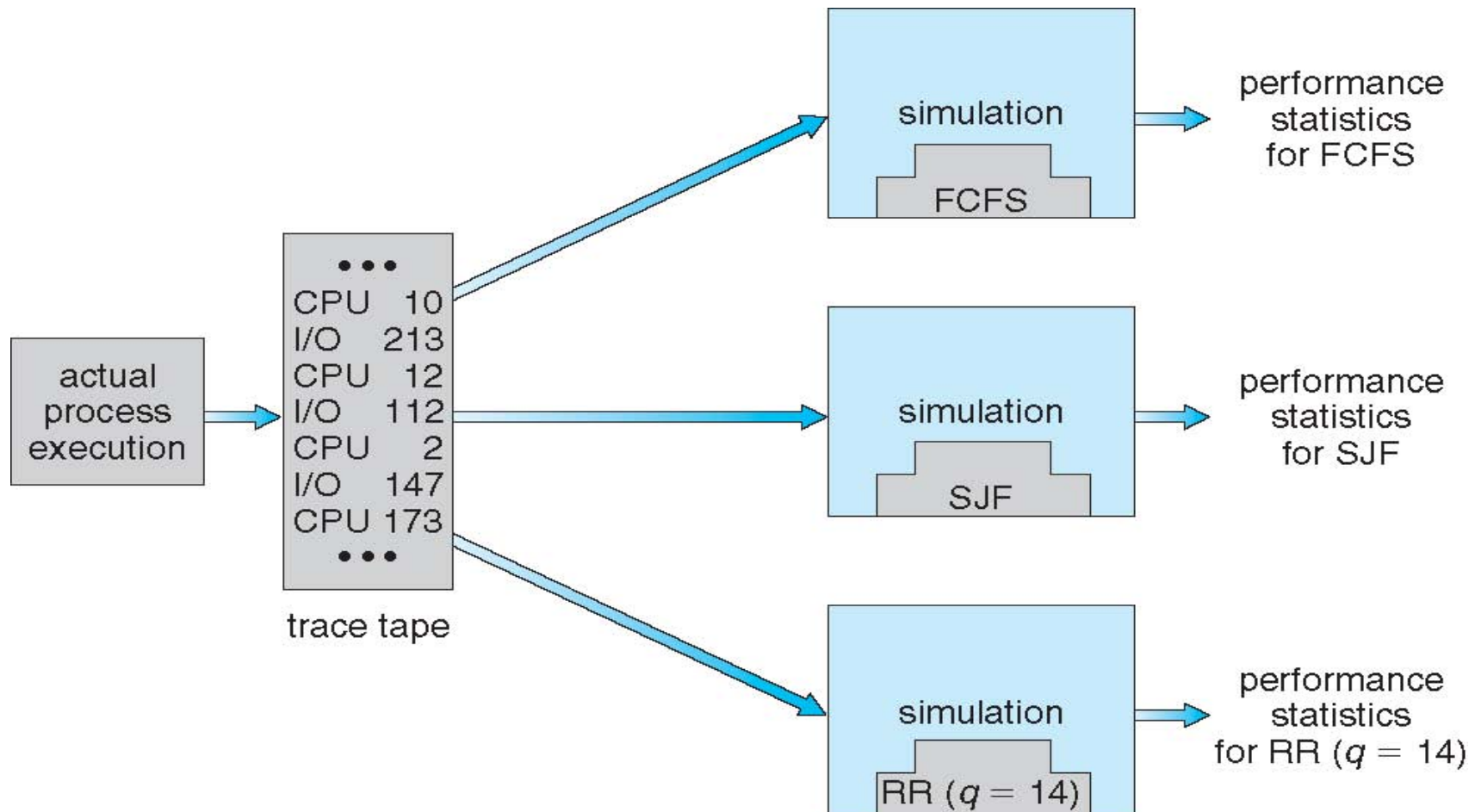
- $n$  = độ dài trung bình của hàng đợi
- $W$  = thời gian chờ trung bình trong hàng đợi
- $\lambda$  = tốc độ trung bình đến hàng đợi
- Định luật Little – ở trạng thái cân bằng, số tiến trình đến hàng đợi bằng số tiến trình rời hàng đợi

$$n = \lambda \times W$$

# Phương pháp mô phỏng

- Sử dụng mô hình được lập trình cho hệ máy tính
- Tham số đồng hồ thời gian
- Thu thập số liệu của thuật toán lập lịch
- Dữ liệu dùng cho mô phỏng được thu thập qua
  - Tạo số ngẫu nhiên theo hàm xác suất
  - Phân bổ số liệu dựa trên hàm toán học hoặc thực nghiệm
  - Theo dõi chuỗi các sự kiện trong một hệ thống thực

# VD đánh giá bằng phương pháp mô phỏng



# Triển khai thuật toán trên hệ thống

- Ưu điểm:
  - Đánh giá chính xác
- Nhược điểm:
  - Giá thành cao
  - Môi trường sử dụng thuật toán có thể thay đổi

# Bài tập 1

Có 5 tiến trình  $P_1, P_2, P_3, P_4, P_5$  với thời gian chạy CPU (ms), thời gian đến và số hiệu ưu tiên như sau:

|       | Thời gian chạy | Số hiệu ưu tiên | Thời gian đến |
|-------|----------------|-----------------|---------------|
| $P_1$ | 2              | 2               | 0             |
| $P_2$ | 1              | 1               | 1             |
| $P_3$ | 8              | 5               | 3             |
| $P_4$ | 4              | 3               | 4             |
| $P_5$ | 5              | 4               | 5             |

Với mỗi thuật toán lập lịch sau hãy vẽ biểu đồ Gantt và tính thời gian chờ trung bình, thời gian phản hồi trung bình, thời gian hoàn thành trung bình, số lần chuyển trạng thái của CPU

- Round Robin với quantum = 2 (ms)
- SJF cho phép dừng
- Lập lịch theo số hiệu ưu tiên cho phép dừng (biết rằng tiến trình có số hiệu ưu tiên cao hơn sẽ được chạy trước)



# Bài tập 2

Vẽ biểu đồ Gantt và tính thời gian chờ trung bình, thời gian hoàn thành trung bình, thời gian phản hồi trung bình cho các tiến trình khi sử dụng thuật toán hàng đợi đa cấp.

| Hàng đợi            | Tiến trình | Thời gian chạy | Thời gian đến | Thuật toán           |
|---------------------|------------|----------------|---------------|----------------------|
| Hàng đợi trước số 1 | P1         | 50             | 0             | RR                   |
|                     | P2         | 15             | 30            | quantum=20           |
|                     | P3         | 45             | 30            |                      |
| Hàng đợi trước số 2 | P4         | 40             | 0             | SJF cho phép<br>dừng |
|                     | P5         | 10             | 120           |                      |
| Hàng đợi sau        | P6         | 30             | 60            | FCFS                 |
|                     | P7         | 20             | 130           |                      |

Biết rằng trình tự ưu tiên các hàng đợi như sau: Hàng đợi trước số 1, Hàng đợi trước số 2, Hàng đợi sau.

# Tìm hiểu thêm

- Lập lịch cho luồng (Thread scheduling)
- Lập lịch cho hệ đa vi xử lý
- Lập lịch cho CPU thời gian thực (Real time CPU scheduling)